

HARMONIX - APP DEVELOPMENT PROJECT

INTRODUCTION

Vikash P

22f3001758@ds.study.iitm.ac.in

I am a second-year student studying Computer Science Engineering at SRM Institute of Science and Technology (Ramapuram) and currently at the diploma level in IITM BS degree. I have titled this project as **Harmonix**.

DESCRIPTION

The primary goal of this project is to develop a music-streaming web application using VueJS and Flask, a lightweight Python web framework. The application aims to provide an efficient platform for users to listen to songs created by various artists and create personalized playlists. The system will have three main user roles: Admin, Users, and Creators. The Admin will have privileges to view overall statistics and performance metrics, including song performances and member counts (users and creators). Users will be able to log in, search for songs and creators, add songs to their playlists, and play songs along with their lyrics. Creators will have the ability to add new songs/lyrics, edit existing song details, and delete songs as needed.

TECHNOLOGIES USED

- *Flask: Micro web framework for backend API and database communication.*
- *VueJS: Frontend for interactive experience.*
- *Jinja2: Renders dynamic content in HTML templates with Flask.*
- *SQLite: Lightweight and easy-to-setup database.*
- *Redis and Celery: Handle asynchronous batch jobs for efficient task execution.*
- *JWT: Token-based authentication for secure user login and access control.*
- *Bootstrap: Enhances UI responsiveness and cross-device compatibility.*

DB SCHEMA DESIGN

<https://drive.google.com/file/d/1veIWqbVUTSivl2opD6dl0eGXStaySKCV/view?usp=sharing>

API DESIGN

The Music Streaming App's API serves as a robust foundation, providing secure access for Admin, User, and Creator logins. It focuses on Song Management, enabling CRUD operations to efficiently handle songs, including details such as creator, genre, and lyrics. Users can explore, play, and manage songs seamlessly. The API enhances the overall music streaming experience by simplifying song-related operations and ensuring a user-friendly interface within the application.

ARCHITECTURE AND FEATURES

1. **User Authentication:** The app supports multi-user login with distinct roles for Admin, User, and Creator.
2. **User Functionality:** Users can play/pause songs, adjust volume settings, read lyrics, and create and manage personalized playlists, tailoring their music experience.
3. **Creator Functionality:** Creators possess the ability to contribute to the platform by adding new songs, including lyrics and associated details. They also have the option to edit song lyrics, allowing for continuous refinement of content.
4. **Admin Functionality:** Administrators gain access to comprehensive app statistics, providing insights into top-performing songs, creators, and other relevant metrics.
5. **Additional Features:** The app incorporates a robust search functionality, enabling users to discover songs and creators based on various criteria easily. Furthermore, the automatic display of new songs ensures a constantly refreshed and dynamic content experience.
6. **Backend Jobs:** The application supports backend jobs, including daily reminders through SMS/Email, and monthly activity reports.

In summary, the Music Streaming App strives to deliver a seamless and user-friendly music experience. Using Flask for the backend API and VueJS for the frontend, the application effectively handles multi-user authentication, song categorization, and product data management. The project's well-organized structure, adherence to MVC architecture, and careful implementation of

features collectively contribute to the creation of a practical and user-centric music streaming solution.

Demo Video link:

https://drive.google.com/file/d/1ij9Go_Uwvm9_sGZaMoB4b1XkDGySrAeq/view?usp=sharing